CANTINA

Solana Foundation: Subscriptions

Security Review

Cantina Managed review by:
R0bert, Lead Security Researcher
Sujith Somraaj, Lead Security Researcher

June 26, 2026

Contents

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

The Solana Foundation is a non-profit foundation based in Zug, Switzerland, dedicated to the decentralization, adoption, and security of the Solana ecosystem.

From Jun 16th to Jun 17th the Cantina team conducted a review of subscriptions on commit hash 38b88beb. The team identified a total of **51** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	5	5	0
Low Risk	24	21	3
Gas Optimizations	0	0	0
Informational	22	12	10
Total	**51**	**38**	**13**

The Cantina Managed team reviewed Solana Foundation's subscriptions holistically on commit hash 2d7b45bd and concluded that all findings were addressed and no new vulnerabilities were identified.

2.1 Scope

The security review had the following components in scope for subscriptions on commit hash 38b88beb:

```
program
├── build.rs
├── Cargo.toml
└── src
    ├── constants.rs
    ├── entrypoint.rs
    ├── errors.rs
    ├── event_engine.rs
    ├── lib.rs
    ├── events
    │   ├── fixed_transfer.rs
    │   ├── mod.rs
    │   ├── recurring_transfer.rs
    │   ├── subscription_cancelled.rs
    │   ├── subscription_created.rs
    │   ├── subscription_resumed.rs
    │   └── subscription_transfer.rs
    ├── instructions
    │   ├── cancel_subscription.rs
    │   ├── close_subscription_authority.rs
    │   ├── create_fixed_delegation.rs
    │   ├── create_plan.rs
    │   ├── create_recurring_delegation.rs
    │   ├── delete_plan.rs
    │   ├── emit_event.rs
    │   ├── initialize_subscription_authority.rs
    │   ├── mod.rs
    │   ├── resume_subscription.rs
    │   ├── revoke_abandoned_delegation.rs
    │   ├── revoke_delegation.rs
    │   ├── revoke_subscription_authority.rs
    │   ├── subscribe.rs
    │   ├── transfer_fixed_delegation.rs
```

```
│   ├── transfer_recurring_delegation.rs
│   ├── transfer_subscription.rs
│   ├── update_plan.rs
│   └── helpers
│       ├── delegation.rs
│       ├── mod.rs
│       ├── plan.rs
│       ├── program.rs
│       ├── system.rs
│       ├── token.rs
│       ├── traits.rs
│       ├── transfer_data.rs
│       ├── transfer_hook_util.rs
│       ├── transfer_utils.rs
│       └── transfer_validation.rs
└── state
    ├── common.rs
    ├── fixed_delegation.rs
    ├── header.rs
    ├── mod.rs
    ├── plan.rs
    ├── recurring_delegation.rs
    ├── subscription_authority.rs
    ├── subscription_delegation.rs
    └── versioning
        ├── core.rs
        ├── mod.rs
        └── v1_to_v2.rs
```

3 Findings

3.1 Medium Risk

3.1.1 Creation signatures have no on-chain freshness deadline

Severity: Medium Risk

Context: create_fixed_delegation.rs#L14-L40, create_recurring_delegation.rs#L19-L34, subscribe.rs#L38-L45

Description: Several user-signed creation instructions bind the user to important terms, but not to the latest time those terms may be used on-chain. `CreateFixedDelegationData` binds the delegator to a nonce, amount, expiry timestamp and `SubscriptionAuthority` generation. `CreateRecurringDelegationData` binds the delegator to the recurring amount, period length, optional start time, expiry timestamp and authority generation. `SubscribeData` binds the subscriber to the plan terms and authority generation.

None of those payloads include a `valid_until_ts`, `max_creation_ts`, `max_start_ts` or equivalent freshness deadline. If the signed transaction remains usable through a durable nonce, relayer-controlled signing flow, merchant-controlled frontend, wallet queue or another delayed-submission path, the program can create the delegation or subscription much later than the user likely intended.

The stale-creation effect is especially visible in three cases. A fixed delegation with `expiry_ts == 0` can create a non-expiring allowance whenever the delayed transaction lands. A recurring delegation with `start_ts == 0` uses the landing timestamp as the first period start, so a delayed transaction can create a fresh full-period allowance shortly before the signed expiry. A subscribe transaction starts the subscription at the landing timestamp as long as the plan snapshots still match, so an old subscription authorization can become freshly billable later.

Consequently, a party that can hold and later submit one of these signed creation transactions can turn old user intent into a new spendable state. Delegatees can receive fixed or recurring pull rights later than the delegator expected and merchants or authorized pullers can create a billable subscription long after the subscriber's original interaction. Existing term and authority-generation checks prevent some stale-state mismatches, but they do not tell the program whether the user's creation authorization is still fresh.

Recommendation: Add an explicit freshness deadline to each user-signed creation payload. A common field such as `valid_until_ts` works or each instruction can use a more specific name such as `max_creation_ts` for delegation creation and `max_start_ts` for subscription creation.

Reject the instruction when `Clock::unix_timestamp` is past the signed freshness deadline. Keep the existing plan-term, expiry and authority-generation checks, but do not treat them as a substitute for transaction freshness.

Apply the check to all creation variants, including fixed delegations with `expiry_ts == 0`, recurring delegations with `start_ts == 0` and subscriptions to perpetual or still-active plans. For recurring delegations, keep `expiry_ts` as the end of the delegation's allowed lifetime; it should not also serve as the latest acceptable creation time.

Solana Foundation: Fixed in commit d71f709.

Cantina Managed: Fix verified. Solved via documentation.

3.1.2 Owners can remove plan expiry after subscribe

Severity: Medium Risk

Context: update_plan.rs#L96-L110

Description: `subscribe` binds the subscriber to the plan mint, amount, period length, plan creation timestamp and `SubscriptionAuthority` generation. It does not bind the subscriber to the plan's `end_ts`, even when the plan is finite at the time of subscription.

`update_plan` lets the active plan owner later set `end_ts` to `0`, which means the plan no longer has an end date. `transfer_subscription` then checks the live plan state and treats the subscription as still active, because the subscriber's account only snapshots the plan terms, not the original expiry.

Consequently, a subscriber can agree to a finite plan and later become billable beyond the end date they saw when they subscribed. The plan owner or an authorized puller can keep collecting future full periods after removing the expiry, unless the subscriber notices the change and cancels. Wallets and marketplaces that display `end_ts` as a fixed subscriber protection can also mislead users if they do not monitor later plan updates.

This is different from charging a full amount for a partial final period. In that case, the original plan end still exists. Here, the owner removes the end date entirely, so later full billing periods remain collectible under the live plan.

Recommendation: If plan expiry is meant to limit subscriber authorization, bind it into the subscription itself. Add an `expected_end_ts` field to `SubscribeData`, reject subscription creation if the live plan expiry does not match what the subscriber signed and store the subscribed expiry in `SubscriptionDelegation`.

`transfer_subscription` should then enforce the stored subscriber expiry as a cap even if the live plan later extends the expiry or removes it by setting `end_ts` to `0`.

If expiry removal is intended to remain owner-controlled, the UI and SDK should present `end_ts` as mutable merchant administration rather than as a subscriber protection.

Solana Foundation: Fixed in commit 4d623cd.

Cantina Managed: Fix verified. The patch makes a finite `plan.data.end_ts` monotonic: `update_plan` now rejects any update where the existing end timestamp is non-zero and the new value is either `0` or greater than the existing value, so a merchant can only shorten a finite plan and cannot remove or extend the subscriber-visible expiry. `transfer_subscription` still uses the live plan expiry for both the top-level expired-plan check and the recurring-transfer cap, so preserving the finite live expiry closes the original path where a subscriber agreed to a finite plan and was later made billable indefinitely.

3.1.3 Authority generation can revive old delegations

Severity: Medium Risk

Context: initialize_subscription_authority.rs#L87-L131

Description: `initialize_subscription_authority` only assigns a new `init_id` when the `SubscriptionAuthority` account is first created. If the account already exists, the instruction reapproves the same authority as the token-account delegate while keeping the old `init_id`. `revoke_subscription_authority` clears the token-account delegate approval, but it does not rotate the program generation or invalidate existing delegation records.

The generation value also comes from the current slot when the account is created. If a user closes and recreates the authority in the same slot, the recreated account can receive the same generation as the closed account. Old fixed delegations, recurring delegations and signed subscribe data that compare only against `init_id` can still match the new live authority.

Consequently, revocation and recreation can behave like a pause instead of a durable invalidation. Old delegatees, merchants or pullers can regain transfer authority after the user reapproves the same authority and same-slot recreation can leave stale delegation or subscription intent valid even though the user expected the authority lifetime to have changed.

Recommendation: Use a durable generation that changes every time the authority is revoked, reapproved, closed or recreated. The generation should not repeat within a slot and should not reset by closing and recreating the authority account.

Continue storing the generation in delegation and subscription records, but reject transfers and stale subscribe data whenever the stored generation differs from the current authority generation. If reactivation of old records is intended, expose it as a separate explicit instruction and make clients present that behavior clearly.

Solana Foundation: Fixed in commit a6ab692. The same-slot `init_id` reuse caveat is documented in commit 2111261.

Cantina Managed: Solved. The close-on-revoke remediation addresses the primary revival vector, while the durable-generation / same-slot limitation is documented and accepted as residual behavior.

3.1.4 Sunset freezes compromised puller authority

Severity: Medium Risk

Context: update_plan.rs#L92-L94

Description: `update_plan` rejects every update after a plan enters `Sunset`. That includes updates that would only remove an existing puller from the plan's puller list.

This matters because `transfer_subscription` still allows existing subscription pulls for sunset plans. It checks the live plan expiry, then authorizes the caller with `plan.can_pull(accounts_struct.caller.address())`. Therefore every puller stored at the moment of sunset remains authorized until the plan reaches `end_ts`.

If a third-party billing operator is compromised after sunset, the plan owner cannot remove that operator from the plan. The compromised puller can continue initiating subscription charges for the remaining sunset period. When the plan has open destinations or when an allowed destination is controlled by the puller, this can become direct value extraction. Even with merchant-only destinations, the owner has no plan-level incident response tool to stop unwanted charges before expiry.

Recommendation: Allow limited updates after `Sunset` that only reduce pull authority. Specifically, permit the owner to replace the puller array with a subset of the current nonzero pullers.

Continue rejecting reactivation, new puller additions, puller substitutions, expiry extension, metadata changes and any other update that expands authority or changes the plan's public terms. Add a regression test that sunsets a plan with puller `P`, removes `P` and confirms a later `transfer_subscription` signed by `P` fails with `Unauthorized`.

Solana Foundation: Fixed in commit 31ae0e4.

Cantina Managed: Fix verified. Commit 31ae0e4 replaces the previous unconditional `PlanImmutableAfterSunset` rejection with a dedicated sunset update path. The plan owner check still runs first and `assert_sunset_puller_reduction` requires the submitted status to remain `Sunset`, the submitted `end_ts` to equal the current plan `end_ts`, the submitted metadata to equal the current metadata and every nonzero submitted puller to already exist in the current puller array before `plan.data.pullers` is overwritten. This permits owner-driven puller removal while still rejecting reactivation, puller additions, puller substitutions, expiry changes and metadata changes.

The transfer path also reads the live plan at collection time and calls `plan.can_pull(accounts_struct.caller.address())`, which authorizes only the plan owner or a nonzero address currently present in `plan.data.pullers`. After the owner removes a compromised puller from a sunset plan, that puller is no longer in the live authorization set and `transfer_subscription` fails with `Unauthorized`.

3.1.5 Active finite plans freeze during the final period

Severity: Medium Risk

Context: update_plan.rs#L33

Description: `update_plan` applies `validate_plan_end_ts` to every Active plan update before it checks whether the plan already has the same finite `end_ts`. That helper rejects any non-zero end timestamp less than one billing period from the current clock.

Consequently, once an Active finite plan enters its final billing period, the owner cannot submit an update that keeps the existing `end_ts`. Puller removal, metadata updates and an Active-to-Sunset transition with the same end timestamp all fail before the mutable fields are written.

This leaves the owner without the intended incident-response control during the period where a compromised puller can still collect from existing subscriptions. The already-Sunset reduction logic does not help because the plan must first be updated into Sunset and that update is rejected while the plan is still Active.

Recommendation: Apply the one-period horizon check only when an Active update sets a new finite end timestamp or shortens the existing one. If `data.end_ts == plan.data.end_ts`, allow puller removal, metadata updates and Active-to-Sunset transition to proceed.

Keep rejecting end timestamp removal or extension for plans that already have a finite end. Add regression tests for unchanged-end puller removal and unchanged-end Sunset transition when the current time is inside the final billing period.

Solana Foundation: Fixed in commit 9f92282.

Cantina Managed: Fix verified. The patch changes `update_plan` so unchanged finite `end_ts` values no longer pass through the one-period horizon check. `UpdatePlanData::validate` now only validates the status byte, `update_plan` first rejects expired plans and finite-end removal or extension and then calls `validate_plan_end_ts` only when `data.end_ts != old_end_ts`.

That preserves the intended protections: a finite plan still cannot clear or extend its end timestamp and a newly set or shortened end timestamp must still be at least one billing period away. But if the update keeps the existing `end_ts`, the owner can remove pullers, edit metadata, or move an Active plan to Sunset during the final billing period and at the exact end timestamp while the plan is not yet expired.

3.2 Low Risk

3.2.1 Lifecycle signatures lack state binding

Severity: Low Risk

Context: cancel_subscription.rs#L24-L84, close_subscription_authority.rs#L44-L80, mod.rs#L306-L336, revoke_delegation.rs#L73-L155

Description: Several lifecycle instructions rely on a signer and the current account state, but the signed instruction data does not bind the user's authorization to a specific account generation, expected state or deadline. `CancelSubscription`, `CloseSubscriptionAuthority` and direct-delegation `RevokeDelegation` are dispatched as no-data instructions. Their handlers then authorize the action against whatever account currently exists at the supplied PDA.

Those PDAs can be reused after the old account is closed. A signed instruction intended for an earlier subscription, authority, fixed delegation or recurring delegation can therefore fit a later account created with the same seeds. `CancelSubscription` also computes the cancellation expiry from the time the instruction executes, so a cancellation signed near a billing boundary can take effect later than the subscriber intended if submission is delayed.

Consequently, a delayed transaction can cancel a recreated subscription, close a recreated `SubscriptionAuthority`, revoke a later delegation that reused a nonce or leave one extra subscription period billable. The attacker or failure case needs access to a still-usable signed transaction, such as through a relayer, queued wallet submission, durable nonce or merchant-controlled frontend.

Recommendation: Add explicit freshness and state-binding data to each affected instruction. Useful fields include `valid_until_ts`, an expected authority or subscription generation, an expected account discriminator, an expected nonce, an expected current cancellation state and for cancellation a maximum acceptable `expires_at_ts`.

Reject instructions when the live account does not match the signed state or when the current clock is past the signed deadline. If account reuse should remain possible, include a non-repeating generation in the account state or PDA seeds so a signature for one account lifetime cannot authorize a later lifetime.

Solana Foundation: Fixed in commit d71f709.

Cantina Managed: Fix verified. Solved via documentation.

3.2.2 IDL omits conditional trailing accounts

Severity: Low Risk

Context: mod.rs#L49

Description: The Codama account annotations publish fixed account lists for instructions whose handlers accept additional accounts after the documented list. `InitSubscriptionAuthority`, `CreateFixedDelegation`, `CreateRecurringDelegation` and `Subscribe` can use a trailing writable signer as the rent payer. `CloseSubscriptionAuthority` can require a trailing writable receiver when the stored payer is not the user. `RevokeDelegation` requires a trailing `plan_pda` for subscription delegations and can also require a trailing rent receiver.

The generated TypeScript and Rust clients follow the published IDL, so they do not expose these conditional accounts as named inputs. A generic IDL consumer can therefore build only the simple variants, even though the program supports sponsor-funded creation, sponsor-funded close and subscription revocation with plan validation.

Consequently, sponsors, subscribers, wallets and SDK users can fail to use supported account states unless they already know the undocumented remaining-account layout. Transactions built directly from the official IDL can miss required accounts, charge the wrong payer or fail with missing-account and authorization errors despite matching the public interface artifact.

Recommendation: Expose every conditional trailing account in the public interface. If Codama cannot model these accounts directly, add official generated-client overlays that name the optional payer, sponsor receiver, subscription `plan_pda` and revoke receiver accounts.

Document the exact account layout for each affected instruction and add release checks that construct sponsor-funded creation, sponsor-funded close and subscription revoke transactions from the published client surfaces.

Solana Foundation: Acknowledged.

Cantina Managed: Not fully fixed. Commit 3cf6269 partially addresses the finding by adding omitted optional Codama accounts for the sponsor `payer` on `initSubscriptionAuthority`, `createFixedDelegation`, `createRecurringDelegation` and `subscribe` and for the rent `receiver` on `closeSubscriptionAuthority`. The original `revokeDelegation` interface gap remains.

3.2.3 Transfer hook helper rejects valid hooks

Severity: Low Risk

Context: transfer_hook_util.rs#L24-L137

Description: The local Token-2022 transfer-hook helper is stricter than the Token-2022 hook interface. It rejects every active hook unless the hook's `extra-account-metas` validation PDA is supplied, even though Token-2022 can invoke a hook that only needs the base transfer accounts. The same helper also caps forwarded hook accounts at the fixed static CPI buffer size, which blocks hooks that require more resolved extra accounts than this local limit allows.

Both cases are compatibility failures in the subscriptions program rather than token-program failures. The mint and hook configuration can be valid, but delegated fixed, recurring and subscription transfers fail before Token-2022 gets to execute the hook under its normal rules.

Consequently, otherwise valid Token-2022 mints can be unusable through this program. Token issuers that use base-only hooks or hooks with larger policy account sets cannot rely on delegated subscription transfers, even though direct Token-2022 callers may support the same mint configuration.

Recommendation: Match Token-2022's transfer-hook account handling. Allow base-only hooks without requiring an `ExtraAccountMetaList` validation PDA. When extra accounts are present, forward the resolved accounts through a CPI helper that supports the intended account limit.

If the program intentionally supports only a narrower subset of hooks, document the exact boundary in the public docs and generated clients. Client-side validation should fail early with the same limit the program enforces.

Solana Foundation: Fixed across commits a06ecd6 and 0ed6d3c.

Cantina Managed: Fix verified for the on-chain transfer-hook helper. Commit `a06ecd6` removes the local `ExtraAccountMetaList` validation-PDA guard from `invoke_transfer_checked_with_hook` and forwards caller-supplied hook accounts directly into the Token-2022 `TransferChecked` CPI. Commit `0ed6d3c` removes the remaining fixed static CPI account buffer by switching from `invoke_signed_with_bounds` to `invoke_signed_with_slice`; the helper now builds the CPI account metas and account view slices dynamically from `remaining`, so the previous 60-account local cap is no longer present.

3.2.4 Transfer events omit the credited token account

Severity: Low Risk

Context: fixed_transfer.rs#L19-L24, recurring_transfer.rs#L19-L28, transfer_utils.rs#L153-L161

Description: The transfer events record the receiver owner, but not the exact destination token account that received the tokens. This is only unambiguous when the destination is always the receiver's canonical token account or can otherwise be derived from the event fields.

The transfer helper enforces that the source is the delegator's canonical token account. For the destination, it only checks that the account uses the requested mint. A valid fixed, recurring or subscription transfer can therefore credit an auxiliary token account or a PDA-owned token account while the event only reports the token-account owner.

Consequently, event-only indexers cannot tell which token account actually received the funds. They may reconcile against the receiver owner's canonical token account even when a different token account was credited or they may be unable to compute the credited account's balance delta and withheld fees from the event stream alone. The on-chain transfer remains authorized, but billing, reconciliation, dispute handling and monitoring systems that use transfer events as their source of truth can record incomplete or misleading destination data.

Recommendation: Either enforce that destination accounts are canonical ATAs when the event only emits the receiver owner or add an explicit `receiver_token_account` / `destination_token_account` field to all transfer events. Keep the owner field if it is useful, but document it as the token-account owner rather than the exact credited account. Update event serialization coverage, generated event schemas and downstream decoders so indexers can reconcile the actual destination account.

Solana Foundation: Fixed in commit 071e59e.

Cantina Managed: Fix verified. The fix adds `receiver_token_account` to `FixedTransferEvent`, `RecurringTransferEvent` and `SubscriptionTransferEvent`, serializes it immediately after `receiver` and populates it from the exact writable receiver token account passed to the transfer CPI in all three transfer handlers. The current IDL also exposes `receiverTokenAccount` for all three transfer event schemas, so IDL-based indexers can recover the credited account.

3.2.5 Plan updates emit no lifecycle event

Severity: Low Risk

Context: mod.rs#L163-L165, update_plan.rs#L108-L113

Description: `update_plan` changes mutable plan fields such as `status`, `end_ts`, `pullers` and `metadata_uri`, then returns without emitting a lifecycle event. The instruction schema also only names `owner` and `plan_pda`, so generated clients do not supply the `event_authority` and `self_program` accounts used by the program's self-CPI event pattern.

These fields affect active subscriptions and monitoring assumptions. `end_ts` can be shortened, extended or removed, `status` can move the plan to `Sunset`, `pullers` can add or remove accounts allowed to collect subscription payments and `metadata_uri` controls marketplace-facing plan data.

Consequently, event-driven indexers, marketplaces, billing systems and subscriber alerts can miss important plan changes unless they also poll and diff plan accounts.

Recommendation: Add a `PlanUpdatedEvent` emitted from `update_plan` after the new fields are written. Include the plan PDA, owner, status, end timestamp, updated pullers and either the metadata bytes or a stable metadata hash. Add `event_authority` and `self_program` accounts to the `UpdatePlan` instruction schema so generated clients can supply the self-CPI event accounts.

If event emission is intentionally omitted to save compute, document that plan updates are account-state-only and provide an official polling and diffing strategy for indexers and marketplaces.

Solana Foundation: Fixed in commit f30fea2.

Cantina Managed: Fix verified. `update_plan` now requires `event_authority` and `self_program`, writes the requested plan changes, drops the mutable plan borrow and emits a `PlanUpdatedEvent` through the program's self-CPI event path.

3.2.6 Exact expiry can block final period pull

Severity: Low Risk

Context: transfer_validation.rs#L70-L79

Description: `validate_recurring_transfer` advances recurring-period accounting by computing the latest period boundary as `candidate_start`. When that boundary is at or after a finite `expiry_ts`, the helper skips the rollover entirely instead of moving accounting to the last billable period before expiry.

This can leave `amount_pulled_in_period` from an older period in force at the exact expiry boundary. For subscriptions, `transfer_subscription` treats `plan.end_ts` as inclusive because it only rejects when `current_ts > plan_end_ts`. A pull at the exact plan end can therefore still be allowed by the expiry check, but rejected by stale period accounting if an earlier period was already fully used.

Consequently, an otherwise valid final-period pull can fail with `AmountExceedsPeriodLimit` at the exact inclusive expiry timestamp. This can deny the merchant or authorized puller the final amount the rest of the subscription logic still treats as billable. The same stale rollover rule can also reduce usable allowance for direct recurring delegations during the expiry drift window when the floored period boundary reaches or passes `expiry_ts`.

For subscribers, this creates inconsistent billing behavior at the boundary. Whether the final pull succeeds can depend on whether a previous transfer happened to reset the period state before expiry, rather than only on the plan terms and current timestamp.

For example:

- A merchant creates a subscription plan that allows charging up to 100 USDC per billing period.
- Alice subscribes at timestamp 1000. The first billing period starts at 1000.
- The plan has a finite end timestamp at 1010, and the program treats that exact timestamp as still billable.
- During the first period, the merchant already pulls the full 100 USDC.
- No other pull happens before timestamp 1010, so the subscription still records: `current_period_start_ts = 1000` and `amount_pulled_in_period = 100`.
- At timestamp 1010, the merchant tries to pull a small final amount, for example 10 USDC.
- The expiry check allows the request because the plan only rejects when `current_ts > end_ts`, and here `current_ts == end_ts`.
- But the period rollover logic refuses to advance the accounting because the new period boundary is exactly equal to `expiry_ts`.

- As a result, the program still thinks the current period has already used 100 USDC.
- The final 10 USDC pull fails with `AmountExceedsPeriodLimit`, even though the exact expiry timestamp is otherwise treated as billable.

Recommendation: When a finite expiry blocks `candidate_start`, advance to the greatest period start that is still strictly before `expiry_ts`, then reset `amount_pulled_in_period` if that capped start is newer than the stored start.

Alternatively, make expiry exclusive for recurring accounting and align all subscription expiry checks with that rule. The important property is that exact expiry should not keep stale accounting from an unrelated older period.

Solana Foundation: Fixed in commit 1ff98ec.

Cantina Managed: Fix verified. The vulnerable behavior was that `validate_recurring_transfer` skipped rollover entirely when the floored `candidate_start` landed at or after a finite expiry, leaving an older `amount_pulled_in_period` in force. The fix adds the capped branch that computes the greatest period boundary strictly before `expiry_ts` and resets `amount_pulled_in_period` only when that boundary is newer than the stored start. `transfer_subscription` still treats `plan.end_ts` as inclusive (`current_ts > plan_end_ts`) and passes `plan_end_ts` into this helper, so an exact-end transfer now reaches the corrected capped accounting.

3.2.7 Spent fixed delegations can lock sponsor rent until expiry

Severity: Low Risk

Context: revoke_delegation.rs#L133-L149

Description: `revoke_delegation` uses the same sponsor-close condition for fixed and recurring delegations. When the caller is the recorded sponsor, the instruction reads the delegation's `expiry_ts` and only allows the close if the delegation is effectively expired. It does not also treat a fixed delegation with `amount == 0` as finished.

A fixed delegation is terminal once its remaining `amount` reaches zero. `transfer_fixed_delegation` subtracts each successful transfer from `delegation.amount`, and `validate_fixed_transfer` rejects any later non-zero transfer that exceeds the remaining allowance. When the remaining allowance is zero, the delegation cannot be used to move more tokens. This is different from a recurring delegation, where the current-period budget can reset later.

Consequently, a sponsor who funded rent for a fully spent fixed delegation may be unable to reclaim that rent even though the account is no longer useful. If `expiry_ts == 0`, the sponsor's normal expiry-based recovery never unlocks, so rent remains tied to the account until the delegator signs a revoke or the related `SubscriptionAuthority` later becomes abandoned.

Recommendation: Allow the recorded sponsor to revoke a fixed delegation when `amount == 0`. Keep the existing expiry requirement for recurring delegations, since a recurring delegation with no currently available allowance is not terminal and may become usable again in a later period.

Solana Foundation: Fixed in commit 540db80.

Cantina Managed: Fix verified. `revoke_delegation` now branches on the delegation discriminator and allows the recorded sponsor to close a fixed delegation when `FixedDelegation::amount == 0`, while recurring delegations remain recoverable by the sponsor only through expiry. This matches the terminal lifecycle of fixed delegations because `create_fixed_delegation` rejects an initial zero amount, `transfer_fixed_delegation` only decreases `amount` after `validate_fixed_transfer` accepts a non-zero transfer within the remaining allowance and later fixed transfers are rejected once the remaining amount is exhausted.

3.2.8 Stale subscription authorities lock sponsor-funded subscription rent

Severity: Low Risk

Context: revoke_delegation.rs#L101-L123

Description: `subscribe` supports a separate sponsor payer and stores that payer in the subscription header together with the current `SubscriptionAuthority.init_id`. Subscription transfers later compare the live `SubscriptionAuthority.init_id` against the value stored in the subscription. If the subscriber closes the authority and recreates it in a later slot, the old subscription no longer matches the live authority generation and transfers fail with `StaleSubscriptionAuthority`.

The sponsor cannot close the subscription in that state. The subscription branch of `revoke_delegation` lets the sponsor close only when the subscription has been cancelled and expired, the plan account is closed, the plan terms changed or the plan ended. It does not treat a closed or later-generation `SubscriptionAuthority` as an abandoned subscription, even though the same condition is already terminal for fixed and recurring delegations through `revoke_abandoned_delegation`.

Consequently, a sponsor that funded a perpetual subscription can have the subscription account rent locked indefinitely after the subscriber rotates their authority. The subscriber can still cooperate by cancelling and waiting out the period, but the recorded sponsor has no unilateral recovery once the subscription can no longer bill.

Recommendation: Add a sponsor recovery condition for subscription delegations whose recorded `SubscriptionAuthority` is closed or whose live `init_id` no longer matches the subscription header. The fix can mirror `revoke_abandoned_delegation`, but it must support `SubscriptionDelegation` and verify that the supplied authority account matches the subscription's recorded subscriber and mint before allowing closure back to the stored payer.

Solana Foundation: Fixed in commit 0de29d1.

Cantina Managed: Fix verified. The patch adds `RevokeAbandonedSubscription` and wires discriminator `16` through the instruction parser, entrypoint, IDL and tests. The processor loads the subscription, requires the signer to be the recorded `header.payer`, requires the supplied `plan_pda` to equal `header.delegatee`, reads the mint from that bound live plan, derives the canonical `SubscriptionAuthority` for `(header.delegator, plan.mint)` and closes the subscription to the payer only when that exact authority is closed or its live `init_id` differs from the subscription header. This is the inverse of the transfer-time liveness check that produces `StaleSubscriptionAuthority`, so the sponsor now has a unilateral rent-recovery path exactly when the subscription has become permanently unbillable through authority closure or rotation.

3.2.9 Fixed and recurring revocation bypasses version checks

Severity: Low Risk

Context: revoke_abandoned_delegation.rs#L58-L69, revoke_delegation.rs#L133-L141

Description: `check_and_update_version` is the version gate that prevents the program from reading delegation accounts with an unsupported schema. It rejects newer account versions and requires an explicit migration when an older account cannot be lazily upgraded.

`revoke_delegation` applies that guard only to `SubscriptionDelegation`. The fixed and recurring branches load the account with `load_with_min_size` and authorize closure using the current struct offsets without checking `data[VERSION_OFFSET]`. In the sponsor branch, this means the close decision can read fields such as `expiry_ts` from whatever byte offset the current layout expects.

`revoke_abandoned_delegation` has the same pattern for fixed and recurring accounts. It reads `subscription_authority`, `init_id` and `payer` from the current layout without first proving the account is at the current schema.

Consequently, after a future schema change, migration or downgrade, fixed and recurring delegation accounts can be closed or rejected using stale field offsets rather than the rules for their actual version. The current v1 layout keeps the immediate impact low, but the inconsistent version gate makes account migrations and rollback scenarios more fragile. Users, sponsors and integrators could see closures or recovery failures that do not match the versioning rules enforced by the rest of the program.

Recommendation: Run `check_and_update_version` before every fixed and recurring `load_with_min_size` call in revoke instructions. If old accounts must remain closable without a full migration, make that an explicit `NeedsAccountRecreate` case and decode only a version-stable close header. Do not read version-specific fields such as `expiry_ts` until the account has been migrated to the current schema.

Solana Foundation: Fixed in commit bac72d4.

Cantina Managed: Fix verified. The patch resolves the issue by choosing the explicit version-agnostic recovery design instead of adding `check_and_update_version` to revocation. `revoke_delegation` and `revoke_abandoned_delegation` now load fixed and recurring accounts through `load_for_revoke`, whose loader gates on each account type's frozen `V1_LEN`, validates the discriminator, copies the data into an owned zero-padded current-size buffer and ignores appended trailing bytes. ADR-003 now documents the required invariant: delegation and subscription layouts must be append-only and recovery paths must make close decisions only from V1 fields. The current close decisions satisfy that invariant: they read only V1 header fields (`delegator` , `payer` , `init_id`) and V1 fixed/recurring fields (`subscription_authority` , `amount` , `expiry_ts`), while mutable transfer/cancel/resume paths still call `check_and_update_version` before typed mutable loading.

3.2.10 Token approval loss does not unlock sponsor recovery

Severity: Low Risk

Context: revoke_abandoned_delegation.rs#L79-L91

Description: Sponsor recovery only treats a delegation as abandoned when the recorded `SubscriptionAuthority` account is closed or its `init_id` no longer matches. The recovery checks do not inspect whether the user's canonical token account still delegates to that authority or whether that token account is still usable for delegated transfers.

`revoke_subscription_authority` can clear the SPL Token delegate approval while leaving the `SubscriptionAuthority` account alive with the same `init_id`. After that, transfers through existing fixed, recurring or subscription delegations can no longer succeed because the authority is no longer the token-account delegate. The sponsor recovery paths still reject the recorded sponsor because the authority account itself is live and has the expected generation.

Consequently, sponsor-paid rent can remain locked while the delegation is not currently chargeable. For direct no-expiry delegations, normal expiry-based recovery never becomes available. For sponsor-funded perpetual subscriptions, the sponsor cannot recover unless the subscriber cancels, the plan ends, the plan is closed or the plan is recreated with mismatched terms. The user can later reapprove the authority but while approval is absent the sponsor has no unilateral way to close the unusable account.

Recommendation: Extend sponsor recovery to account for token-account liveness, not only `SubscriptionAuthority` liveness. Recovery should accept the canonical source ATA, token mint and token program, then allow the recorded payer to close when the ATA is missing, not canonical or no longer delegates to the recorded `SubscriptionAuthority`.

If immediate recovery after delegate revocation is too aggressive, add an explicit delay or a user-visible paused state. The important property is that a sponsor-funded delegation that cannot transfer because token approval is gone should not remain outside every sponsor recovery condition indefinitely.

Solana Foundation: Fixed for the program-mediated revoke path in commit a6ab692, with the residual direct SPL Token revoke behavior accepted as an intentional UX tradeoff for now.

Cantina Managed: Fix verified with caveat. The implemented fix makes `revoke_subscription_authority` close the live `SubscriptionAuthority` PDA after clearing the

program delegate. This means the normal user-facing revoke path now also unlocks sponsor recovery because old fixed, recurring and subscription records fail once their recorded authority account is closed.

The remaining direct SPL Token `Revoke` path is understood and accepted by Solana Foundation for now. A token-account owner can still bypass this program and revoke the ATA delegate directly, which clears `delegated_amount` while leaving the `SubscriptionAuthority` PDA live with the same `init_id`. In that state sponsor recovery still returns `Unauthorized`. Solana Foundation explicitly accepted this residual behavior to avoid a worse UX case where a user temporarily revokes and re-approves token approval, but sponsors close live subscriptions during that temporary approval gap.

3.2.11 Defensive loaders reject old delegation accounts after size increases

Severity: Low Risk

Context: core.rs#L68-L73

Description: The delegation loaders expose `load_with_min_size` and `load_mut_with_min_size` helpers that are intended to be safer than exact-size loading after schema changes. They are used by revoke and recovery code that may need to read enough account data to close or recover older delegation accounts.

However, these helpers still pass the current struct `Self::LEN` into `check_min_account_size`. If a future release increases the size of `FixedDelegation`, `RecurringDelegation` or `SubscriptionDelegation`, older accounts that were created with the smaller layout will be shorter than the new `Self::LEN`. The defensive loader will reject them with `InvalidAccountData` before any version-specific prefix parsing can recover the fields needed for closure.

Consequently, a future size-increasing upgrade can turn old delegation accounts into liveness problems for users and sponsors. Accounts that still contain enough stable header data to prove the payer, authority and close destination can be rejected only because they are shorter than the new struct. This makes migrations and downgrade or recovery flows more fragile and can produce generic `InvalidAccountData` failures instead of a clear migration or account-recreation path.

Recommendation: Do not use the current struct size as the minimum size for old-account defensive loading. Add version-specific loaders that parse only the fields needed for closure or recovery from each supported historical layout. For revoke and recovery instructions, decode a stable close header before reading version-specific fields, then require migration or account recreation only when the old layout truly cannot be interpreted safely.

Solana Foundation: Fixed in commit bac72d4.

Cantina Managed: Fix verified. The revoke/recovery loaders for `FixedDelegation`, `RecurringDelegation` and `SubscriptionDelegation` now use frozen first-version minimum sizes (`V1_LEN`) through `load_for_revoke`, copy the account into a zero-padded `Self::LEN` buffer and return an owned value. This means a first-version account remains closable even if a later release appends fields and increases `Self::LEN`. The close/recovery call sites in `revoke_delegation`, `revoke_abandoned_delegation` and `revoke_abandoned_subscription` all use `load_for_revoke`, so they no longer reject recoverable old delegation data solely because it is shorter than the current struct size.

3.2.12 Revoke subscription authority can clear unrelated delegates

Severity: Low Risk

Context: revoke_subscription_authority.rs#L41-L65

Description: `revoke_subscription_authority` validates that `user_ata` is the signer's canonical ATA for `token_mint`, then calls SPL Token `Revoke` without checking which delegate is currently set on that token account.

This means the instruction clears any current delegate, not just the `SubscriptionAuthority` PDA that `initialize_subscription_authority` approved. A user or client can replace the

`SubscriptionAuthority` approval with another delegate and later call this program instruction while trying to disable only the subscription authority. The instruction then clears the unrelated delegate and zeroes its delegated amount.

Consequently, a subscriptions-specific cleanup instruction can unexpectedly remove an approval that belongs to another integration. The instruction does more than its name and documentation imply and wallets or official cleanup flows can clear a valid unrelated delegate on the same token account.

Recommendation: Before invoking SPL Token `Revoke`, derive the expected `SubscriptionAuthority` PDA from `user` and `token_mint` and inspect the token account's current delegate field. If no delegate is set, return success or a clear no-op error. If a different delegate is set, reject the instruction instead of clearing it.

Solana Foundation: Fixed in commit 642c2ca.

Cantina Managed: Fix verified. The original vulnerable code invoked SPL Token `Revoke` unconditionally after validating only the ATA owner, mint and canonical ATA address, so any current delegate on that ATA was cleared. Commit 642c2ca added `get_token_account_delegate`, derived the expected `SubscriptionAuthority` PDA from `(user, token_mint)` and gated the token-program `Revoke` CPI on the current delegate matching that PDA. The current code, after the later a6ab692 authority-close change, preserves that safety property: it invokes `Revoke` only when `current_delegate == Some(expected_delegate)` and leaves foreign or absent delegates untouched.

3.2.13 Valid Token-2022 mint padding can block delegated transfers

Severity: Low Risk

Context: transfer_hook_util.rs#L28-L56

Description: `find_extension_value` manually scans Token-2022 mint extension data to determine whether a mint has a transfer hook. The scanner requires the TLV data to end exactly at the last parsed extension or to contain a complete four-byte header for the next extension. Token-2022 can accept valid mint layouts that contain unused trailing padding, including padding used when the extension area is adjusted away from the legacy multisig account length.

`transfer_with_delegate` calls `mint_transfer_hook_program_id` before each delegated transfer. As a result, a valid padded Token-2022 mint can fail with `InvalidToken2022MintAccountData` before the program invokes `TransferChecked`, even though the token program would otherwise accept the mint and process the transfer.

Therefore, fixed, recurring and subscription transfers can be blocked for affected Token-2022 mints.

Recommendation: Match Token-2022's TLV handling when scanning for the transfer hook extension. Treat trailing bytes shorter than a type field and uninitialized zero extension markers, as the end of used TLV data instead of returning `InvalidToken2022MintAccountData`. Prefer using the official Token-2022 extension parser if it is available for the target runtime.

Solana Foundation: Fixed in commit 4b5b31b.

Cantina Managed: Fix verified. The old parser only accepted TLV data that ended exactly on an extension boundary, so valid trailing padding caused `mint_transfer_hook_program_id` to return `InvalidToken2022MintAccountData` before the program invoked Token-2022 `TransferChecked`. The fix changes `find_extension_value` to stop successfully when fewer than a two-byte extension type remains or when it reaches an uninitialized zero extension marker, while still rejecting malformed nonzero partial headers, overlong values and malformed transfer-hook lengths. This matches the relevant valid-padding behavior in Token-2022's TLV walk and preserves hook detection when a `TransferHook` extension appears before padding.

3.2.14 Subscription created event omits sponsored payer

Severity: Low Risk

Context: subscription_created.rs#L10-L19

Description: `SubscriptionCreatedEvent` records the plan, subscriber, token mint and creation timestamp. It does not record the account that funded the new `SubscriptionDelegation`.

This matters because `subscribe` supports an optional sponsor payer through `resolve_optional_payer`. When a sponsor is provided, the program stores that account in `subscription.header.payer` and rent is later returned to the recorded payer when the subscription is revoked. The on-chain state therefore distinguishes subscriber-funded subscriptions from sponsor-funded subscriptions, but the creation event does not.

Therefore, indexers and off-chain billing systems that rely on events cannot tell who funded the subscription account. They may attribute rent costs or future rent refunds to the subscriber even when a sponsor paid for the account. This makes event-only accounting incomplete for a supported subscription mode.

Recommendation: Include the recorded payer in `SubscriptionCreatedEvent`, populated from `accounts_struct.payer.address()` in `subscribe`. This makes the event match the subscription state and lets event consumers distinguish sponsor-funded subscriptions from subscriber-funded subscriptions.

Solana Foundation: Fixed in commit a2b0161.

Cantina Managed: Fix verified. `SubscriptionCreatedEvent` now includes and serializes a `payer` address as the final event field and `subscribe` populates it from `accounts_struct.payer.address()`, the same resolved account used to fund the `SubscriptionDelegation` rent and stored in `subscription.header.payer`. `resolve_optional_payer` requires any sponsor payer to be signer and writable, while the no-sponsor path falls back to the subscriber, so the emitted value matches both supported funding modes. The Codama IDL also exposes `payer` on `subscriptionCreatedEvent`, allowing event consumers to decode the new field.

3.2.15 Subscription transfer event omits authorized puller

Severity: Low Risk

Context: subscription_transfer.rs#L10-L29

Description: `transfer_subscription` allows either the plan owner or a whitelisted puller to initiate a subscription payment. The instruction checks `plan.can_pull(accounts_struct.caller.address())`, but `SubscriptionTransferEvent` does not record that caller. The event records the subscription, plan, subscriber, mint, amount, period accounting and receiver only.

This loses the identity of the account that exercised pull authority. Direct fixed and recurring transfer events include the initiating `delegatee`, but subscription transfer events do not include the equivalent signer. For plans with multiple authorized pullers, the event alone cannot show whether the owner or a specific puller initiated the payment.

Consequently, event-only billing, monitoring, compliance and dispute systems cannot attribute subscription pulls to the actor that signed them.

Recommendation: Add an authorized-puller field to `SubscriptionTransferEvent` and populate it with `accounts_struct.caller.address()` in `transfer_subscription`.

Update the event serializer, generated event schema, documentation and downstream decoders so subscription transfer events preserve the signer that passed `plan.can_pull`.

Solana Foundation: Fixed in commit 1032d39; the generated IDL event schema was registered in commit e290958.

Cantina Managed: Fix verified. `SubscriptionTransferEvent` now appends a `puller` field, serializes that field and `transfer_subscription` populates it from `accounts_struct.caller.address()` after

the same signer has passed `plan.can_pull(...)`. Appending the field preserves the offsets of the pre-existing event fields, while `idl/subscriptions.json` exposes `subscriptionTransferEvent.puller` as a public key for downstream decoders.

3.2.16 Recurring transfer event overstates final period end

Severity: Low Risk

Context: transfer_recurring_delegation.rs#L89-L100

Description: `transfer_recurring_delegation` emits `RecurringTransferEvent.period_end_ts` as `period_start + period_length_s` for every successful recurring delegation transfer. The emitted value is not capped by the delegation's finite `expiry_ts`.

This makes direct recurring delegation events less precise than subscription transfer events. `transfer_subscription` caps its emitted `period_end_ts` by the finite plan end timestamp, but the direct recurring transfer event always reports the nominal period boundary. A transfer near the end of a finite recurring delegation can therefore emit a period end later than the delegation's actual expiry.

Consequently, off-chain billing, reconciliation and monitoring systems can display or account for a paid period that extends past the delegation's real lifetime. Consumers relying on the event stream can overstate the covered service window for the final period.

Recommendation: Cap the emitted direct recurring transfer period end by the delegation expiry, matching the subscription transfer behavior.

```
let end = period_start + period_length_s as i64;
let period_end_ts = if expiry_ts != 0 && end > expiry_ts { expiry_ts } else { end };
```

If consumers need both timestamps, emit separate fields for the nominal period end and the effective expiry-capped period end.

Solana Foundation: Fixed in commit 3d0136f.

Cantina Managed: Fix verified. `transfer_recurring_delegation` now carries the recurring delegation's `expiry_ts` out of the mutable delegation borrow and computes `RecurringTransferEvent.period_end_ts` as the nominal `period_start + period_length_s` capped to `expiry_ts` when the delegation has a finite expiry. This matches the existing subscription-transfer event behavior and directly closes the original event-only overstatement path.

3.2.17 Published event schema can drift from emitted events

Severity: Low Risk

Context: subscriptions.json#L1288, subscription_created.rs#L8-L19, subscription_transfer.rs#L8-L29

Description: The program emits self-CPI events through local `EventSerialize` implementations, but those event structs are not registered as Codama events. The published IDL therefore has an empty event list. At the same time, the architecture documentation describes event payloads manually and does not match the fields that the program actually serializes.

This leaves two separate descriptions of the event interface: private Rust serializers in the program and manually maintained public documentation. Generated clients and IDL-based indexers cannot discover the event schemas, while hand-written decoders can drift from the emitted bytes.

Consequently, off-chain billing, reconciliation, monitoring and dispute systems can miss events or decode them incorrectly. Consumers may expect fields that are never emitted, omit fields needed for period accounting or rely on documentation that no longer matches the event wire format.

Recommendation: Register every emitted event with Codama so `idl/subscriptions.json` and generated clients expose the same schema the program serializes. The registered events should cover the six emitted event types and their discriminators, field order and field types.

Generate or check the public documentation from the same schema source.

Solana Foundation: Fixed in commit e290958. The later `PlanUpdatedEvent` added in commit f30fea2 is also registered in the IDL.

Cantina Managed: Fix verified. The six events covered by the original finding now derive `CodamaEvent` and declare the exact self-CPI wire discriminators: the 8-byte Anchor event tag at offset 0 and the one-byte `EventDiscriminators` value at offset 8. The current codebase has seven emitted event types after `PlanUpdatedEvent` was added and all seven are present in `idl/subscriptions.json` under `program.events` with matching discriminators, field order and field types.

3.2.18 Generated event defaults ignore custom program addresses

Severity: Low Risk

Context: mod.rs#L214-L223

Description: The Codama account annotations for event-emitting instructions hardcode `event_authority` and `self_program` as fixed public keys. The generated TypeScript builders still accept `config.programAddress`, but their default event accounts do not follow that address.

For example, a caller can build a `Subscribe` instruction with a custom program address. The instruction itself uses the custom address, but the generated defaults still fill `eventAuthority` with the published program's event authority and `selfProgram` with the published program address. The on-chain event code then verifies the supplied `event_authority` and performs the event self-CPI through the supplied `self_program`, so those accounts must match the program that is actually executing.

Consequently, custom, cloned or local deployments can build instructions that look valid but fail during event emission unless the integrator manually overrides both event accounts. This affects subscribe, cancel, resume and transfer instructions that emit events. It does not bypass authorization or move funds, but it makes custom deployments harder to test and support.

Recommendation: Generate event defaults from the active program address instead of fixed public keys. `self_program` should default to the same program address used for the instruction and `event_authority` should default to the PDA derived from `event_authority` and that program address. If Codama cannot express that dependency directly in the instruction annotations, add generated-client overlays that resolve these accounts from `programAddress` for every event-emitting instruction.

Solana Foundation: Fixed in commit 290c185.

Cantina Managed: Fix verified. The fix adds a shared Codama visitor in `scripts/event-account-fixups.ts` and applies it from both `scripts/generate-clients.ts` and `scripts/generate-ts-client.ts`.

3.2.19 `TokenAccount2022Account::check` panics (out-of-bounds index) on short Token-2022-owned accounts

Severity: Low Risk

Context: token.rs#L216-L224

Description: In `token.rs`, the validator early-returns `Ok` only when `data_len() == 165`; for any other length it borrows the data and indexes `data[TOKEN_2022_ACCOUNT_DISCRIMINATOR_OFFSET]` (= `data[165]`, line 222) without a length guard.

The sibling `Mint2022Account::check` correctly guards `data.len() <= 165` before the identical access. Because anyone can create an account owned by the Token-2022 program with an arbitrary

sub-166-byte length, passing such an account as `delegator_ata` / `receiver_ata` in any transfer path makes `data[165]` an out-of-bounds slice index, which panics/aborts instead of returning a clean `InvalidToken2022TokenAccountData`.

Recommendation: Mirror the mint validator: before indexing,

```
if data.len() <= TOKEN_2022_ACCOUNT_DISCRIMINATOR_OFFSET
   || data[TOKEN_2022_ACCOUNT_DISCRIMINATOR_OFFSET] !=
   ↪  TOKEN_2022_TOKEN_ACCOUNT_DISCRIMINATOR
{
   return Err(SubscriptionsError::InvalidToken2022TokenAccountData.into());
}
```

Solana Foundation: Fixed in commit 771e913.

Cantina Managed: Fix verified. `TokenAccount2022Account::check` now rejects any non-base-length Token-2022 token account whose borrowed data is `<= TOKEN_2022_ACCOUNT_DISCRIMINATOR_OFFSET` before reading byte 165, so sub-166-byte Token-2022-owned accounts return `InvalidToken2022TokenAccountData` instead of panicking. The shared validator is still used by `revoke_subscription_authority`, by `DelegationTransferAccounts` for fixed and recurring transfers and by `TransferSubscriptionAccounts` for subscription transfers before any later raw token-account reads; those later reads also have their own minimum-length guards.

3.2.20 `resume_subscription` reactivates a subscription with no authority/`init_id` liveness check

Severity: Low Risk

Context: resume_subscription.rs#L24

Description: `subscribe` and every `transfer` enforce `subscription_authority.init_id == header.init_id`, but `resume_subscription`, the only lifecycle write that re-activates a subscription, never loads the `SubscriptionAuthority` and never checks `init_id` (the account is not in its account list).

A subscriber can cancel, then close and re-initialize their `SubscriptionAuthority` (bumping `init_id`), then call `resume`, which succeeds, clears `expires_at_ts` and emits `SubscriptionResumedEvent`, while every subsequent pull fails `StaleSubscriptionAuthority`. This will result in an "zombie-active" subscription plus a misleading lifecycle event that indexers/merchants treat as chargeable (no funds move).

Recommendation: Require the `SubscriptionAuthority` PDA in `ResumeSubscriptionAccounts`, validate owner/PDA and reject when `subscription.header.init_id != subscription_authority.init_id` before clearing `expires_at_ts` and emitting the event.

Solana Foundation: Fixed in commit 7f51c7a.

Cantina Managed: Fix verified. The patched `resume_subscription` account list now requires `subscription_authority`, validates it as a live program-owned `SubscriptionAuthority`, loads the plan mint, checks that the authority belongs to the subscriber, checks that its mint matches the plan mint and rejects with `StaleSubscriptionAuthority` when `authority.init_id != subscription.header.init_id` before clearing `expires_at_ts` or emitting `SubscriptionResumedEvent`.

3.2.21 Subscriber can stop payment while the subscription stays "Active", with no on-chain delinquency state

Severity: Low Risk

Context: revoke_subscription_authority.rs#L45-L65

Description: A subscriber can call `RevokeSubscriptionAuthority` (owner-signed SPL `Revoke`), or freeze/empty/close their source ATA, so all future pulls fail while the `SubscriptionDelegation` remains `expires_at_ts == 0` (Active).

The program has no on-chain delinquency/lapse state and no merchant-visible signal distinguishing "paid" from "approval revoked". Off-chain merchant logic that trusts on-chain "Active" can be gamed into providing service without payment, with no on-chain recourse.

Recommendation: Document that on-chain "Active" is not proof of collectability; consider a last-successful-pull timestamp or delinquency flag (set when a due period's pull fails / a period elapses uncollected) so merchants can detect lapses on-chain.

Solana Foundation: Documented this behavior in b4fdfd2, but won't implement behavior change. It's on the merchant to validate before providing services.

Cantina Managed: Fix verified.

3.2.22 SPL Token multisig owners cannot use subscription authorities

Severity: Low Risk

Context: initialize_subscription_authority.rs#L39

Description: `initialize_subscription_authority` requires `user` to be a transaction signer, then passes that same account as the token authority in the SPL Token or Token-2022 `Approve` CPI. This only works when the token account owner is a normal signer account.

Native SPL Token multisignature accounts are valid token authorities, but the multisig account itself is not expected to sign. The token program authorizes those accounts by receiving the multisig account as authority together with the required member signer accounts. This program does not accept or forward those member signers. The optional trailing account in initialization is interpreted as a payer, so extra signer accounts cannot repair the approval call.

`revoke_subscription_authority` has the same limitation. It requires `user` to sign at `program/src/instructions/revoke_subscription_authority.rs:28`, then calls `Revoke` with no multisig member signers and ignores trailing accounts. Consequently, an ATA owned by a native SPL Token multisig cannot initialize a `SubscriptionAuthority` through this program and cannot use the program's revoke instruction.

Treasury accounts that use native SPL Token multisig ownership are valid token accounts, but they cannot use this subscription authority model unless ownership is moved to a normal signer or a different wallet abstraction.

Recommendation: Separate the rent payer from the token authority account. Keep the existing single-signer case, but also accept trailing multisig member signer accounts when the token authority is a native SPL Token multisig.

When approving or revoking, pass the multisig account as the token authority and forward the member signer metas to the token program. Let SPL Token or Token-2022 enforce the configured signature threshold.

Solana Foundation: Acknowledged. We will not fix this for now. Most users usually use squads or other smart wallet. In the future we will consider adding a new instruction for multi-sigs directly.

Cantina Managed: Acknowledged.

3.2.23 Mandatory event self-CPI can block cancellation at max depth

Severity: Low Risk

Context: cancel_subscription.rs#L76-L81

Description: `CancelSubscription` commits the cancellation state only if the later event emission succeeds. It first sets `subscription.expires_at_ts`, then immediately calls `event_engine::emit_event(...)?`. That helper emits events through a nested invocation back into this program, so cancellation always needs one additional CPI frame even though the state change itself does not need to call another program.

Solana rejects nested invocations once a transaction is already at the maximum call depth. If a subscriber is a PDA controlled by another program and its only authorized way to sign reaches `CancelSubscription` at that depth, the event CPI fails with a call-depth error. The propagated error rolls back `expires_at_ts`, leaving the subscription active. Authorized pullers can continue charging until the subscriber can use a shallower signing arrangement or another lifecycle condition ends the subscription.

`ResumeSubscription` has the same pattern after clearing `expires_at_ts`, so resumption also loses composability at the same boundary. Cancellation is more sensitive because it is the subscriber's way to stop future subscription pulls.

Recommendation: Do not make cancellation or resumption depend on an additional self-CPI for event emission. Prefer an in-program log or event primitive that does not consume another invocation frame for these lifecycle changes.

If the self-CPI format must remain, add a cancellation and resumption fallback that commits the state change when only the event CPI would fail because the call-depth limit was reached. Document how indexers should handle that fallback so event consumers do not treat the missing self-CPI event as proof that the lifecycle change did not happen.

Solana Foundation: Acknowledged. We will not fix it. CPI limit is 4 which seems reasonable and an increase to 8 should be coming fairly soon. Users can still revoke the subscription if they really are stuck cancelling.

Cantina Managed: Acknowledged.

3.2.24 `revokeSubscriptionAuthority` can skip closing the real authority

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: revoke_subscription_authority only verifies that the supplied `subscription_authority` account is the expected PDA when the supplied account is program-owned and has non-empty data. If the transaction supplies a writable system account or any already-closed account in that slot, the instruction can still return success without closing the real `SubscriptionAuthority` PDA.

This weakens the fix for Token-2022 mints whose `PermanentDelegate` is the real `SubscriptionAuthority` PDA. A malformed revoke can clear the ordinary token-account delegate while leaving the real authority account live. Existing fixed, recurring or subscription delegation records can then continue to call the transfer entrypoints. The transfer helper loads the still-open authority account and signs the Token-2022 CPI and Token-2022 accepts the signer through the mint-level permanent delegate even though the token account delegate is gone.

Recommendation: Always require the supplied `subscription_authority` account address to equal `SubscriptionAuthority::find_pda(user, token_mint).0`, even when the account is closed or system-owned. After that address check, keep the existing close behavior for open program-owned accounts.

Solana Foundation: Fixed in commit 84b689a.

Cantina Managed: Fix verified. The fix changes `revoke_subscription_authority::process` so the supplied `subscription_authority` account must always equal the canonical `SubscriptionAuthority::find_pda(user, token_mint).0` address before the instruction decides whether to close the account. This removes the bypass where a writable system account or unrelated already-closed account could be supplied in the authority slot, allowing the instruction to return success while leaving the real authority PDA open. The fix also adds

`revoke_subscription_authority_rejects_spoofed_authority_account`, which supplies a non-canonical authority account and expects `InvalidSubscriptionAuthorityPda` while confirming the real authority PDA remains open after the rejected transaction.

3.3 Informational

3.3.1 Token-2022 permanent delegate can bypass authority revoke

Severity: Informational

Context: transfer_utils.rs#L177-L210

Description: `transfer_with_delegate` signs token transfers with the user's `SubscriptionAuthority` PDA after checking the live authority account, source token account, mint and stored generation. `revoke_subscription_authority` only clears the normal per-account delegate approval from the user's token account.

For Token-2022 mints, that is not enough when the mint's `PermanentDelegate` extension is set to the same `SubscriptionAuthority` PDA. Token-2022 can authorize the transfer through the mint-level permanent delegate even if the user's token account no longer has a delegate and its delegated amount is zero.

Consequently, a user-facing revoke may not actually pause delegated spending for this Token-2022 mint configuration. Old fixed, recurring subscription delegations can remain usable immediately after `RevokeSubscriptionAuthority`, as long as the `SubscriptionAuthority` account is still live and the protocol's own delegation checks pass. Delegatees and merchants tied to those old records can continue pulling tokens even though the user's token account shows no normal delegate approval.

Recommendation: Do not treat token-account `Revoke` as sufficient to pause this program's transfer authority for Token-2022 mints whose permanent delegate is the `SubscriptionAuthority` PDA.

Consider rejecting authority initialization or delegated transfers when the mint has `PermanentDelegate` set to the derived `SubscriptionAuthority` PDA, unless the user explicitly opts into that behavior. Alternatively, make `revoke_subscription_authority` rotate or close the program authority generation so old delegation records fail before the token CPI, regardless of Token-2022 permanent-delegate authorization.

Solana Foundation: Fixed across commits a6ab692 and 84b689a.

Cantina Managed: Fix verified. Commit a6ab692 changes the canonical `RevokeSubscriptionAuthority` path so it clears the ordinary token-account delegate when it is the expected PDA and then closes the live `SubscriptionAuthority` account. This addresses the root permanent-delegate issue through the alternate remediation from the recommendation: old fixed, recurring and subscription records fail at the program authority check before the Token-2022 CPI can use mint-level permanent-delegate authorization.

Commit 84b689a fixes the remaining bypass from the prior Cantina verification by requiring the supplied `subscription_authority` account address to equal `SubscriptionAuthority::find_pda(user, token_mint).0` before the close decision. A transaction can no longer supply a writable system account or unrelated already-closed account in the authority slot to make revoke return success while leaving the real PDA live.

3.3.2 Subscriptions can charge full amount for partial final period

Severity: Informational

Context: transfer_validation.rs#L70-L88

Description: `validate_recurring_transfer` gives a subscription the full per-period allowance as long as the current billing period starts before the plan expires. It does not check whether the full billing period fits inside the remaining plan lifetime.

This means any final partial period is treated like a complete period for payment purposes. `subscribe` only rejects a finite plan after it has already expired, so a subscriber can join shortly before plan expiry and still be charged the full period amount. `update_plan` validates a new `end_ts` against the update

timestamp, not against each subscriber's next billing boundary, so an existing subscriber can also enter a final period with only a small amount of time left before the updated plan end.

Consequently, subscribers can pay a full period price for a very small remaining service window near plan expiry. Merchants and authorized pullers can collect the full `amount_per_period` even when the finite plan cannot provide a full period of service and marketplaces or support teams may see billing records that are hard to reconcile with the displayed plan end date.

The transfer event caps `period_end_ts` at the plan end timestamp, so the emitted event can show the truncated billing window. The token amount transferred is still the full `amount_per_period`.

Recommendation: Do not grant a full subscription period allowance when the period cannot complete before `plan_end_ts`. For subscriptions with a finite plan expiry, reject transfers unless `current_period_start_ts + period_length_s <= plan_end_ts` or explicitly prorate the allowed amount for the final partial period.

Also apply the same full-period-remaining check when subscribing to a finite plan. This prevents a new subscription from starting inside the final partial period.

Solana Foundation: Acknowledged. This is by design. Users agree to the terms set in the Plan, this includes the plan end ts. We don't want to enforce that at the contract level.

Cantina Managed: Acknowledged.

3.3.3 Self-transfers consume allowance without moving tokens

Severity: Informational

Context: transfer_utils.rs#L167-L210

Description: `transfer_with_delegate` checks that the source token account is the delegator's canonical ATA and that the destination token account uses the requested mint. It does not reject the same token account being passed as both source and destination.

SPL Token and Token-2022 accept self-transfers as successful no-ops. They validate the authority, then return without moving tokens. The subscriptions program updates fixed delegation allowance, recurring period accounting or subscription period accounting before it calls `transfer_with_delegate`. Therefore a self-transfer can consume allowance or mark a billing period as pulled even though no token balance changed.

For fixed and recurring delegations, the authorized delegatee can burn their own allowance without receiving funds. For subscriptions, an authorized puller can consume the subscriber's period allowance without paying the merchant when the plan's destination rules allow the subscriber's token account or allow any destination. This breaks the invariant that consumed allowance corresponds to tokens leaving the delegator's account.

Therefore:

- For subscribers, allowance or period budget can be consumed even though no merchant account receives tokens.
- For merchants and pullers, billing state can show a charge consumed while their receiver balance did not increase.
- For indexers and support teams, they must reconcile a successful transfer event with no meaningful balance movement.

Recommendation: Reject self-transfers in the shared transfer helper before invoking the token program.

```
if accounts.delegator_ata.address() == accounts.to_ata.address() {
    return Err(SubscriptionsError::UnauthorizedDestination.into());
}
```

Alternatively, add a dedicated self-transfer error if the existing destination error is too broad. Add regression tests for fixed, recurring and subscription transfers that pass the delegator ATA as the receiver. The expected result should be an error and unchanged delegation or subscription accounting.

Solana Foundation: Acknowledged. Self transfer would be the problem of the delegatee, if they self transfer it can be considered their fault. We want to keep the program as open as possible and let merchant decide on rules.

Cantina Managed: Acknowledged.

3.3.4 Sponsor-funded authority rent has no recovery

Severity: Informational

Context: close_subscription_authority.rs#L20-L51

Description: `initialize_subscription_authority` can record a sponsor as `SubscriptionAuthority.payer` when the sponsor funds the authority account. The stored payer is the account that receives rent when the authority is eventually closed.

However, `close_subscription_authority` requires the `user` account to sign and checks that the stored authority owner matches that signer. If the stored payer is different from the user, the instruction only sends rent to the sponsor when the user also supplies the sponsor as the receiver. The sponsor is the rent recipient, but it is not allowed to initiate the close.

Consequently, a sponsor that funded a `SubscriptionAuthority` cannot recover the rent if the user disappears, refuses to sign or simply never closes the account.

Recommendation: Either remove sponsor support for `SubscriptionAuthority` creation or document that this rent is a non-recoverable subsidy unless the user cooperates. If sponsor recovery is intended, add a sponsor-controlled close instruction with explicit safety checks, such as requiring that the token delegate no longer points to the authority and that no active dependent subscriptions or delegations remain.

Solana Foundation: Fixed in commit 84e0e9b.

Cantina Managed: Fix verified. The implemented remediation follows the issue's documentation option: `docs/001-program-architecture.md` now states that only the user can close a `SubscriptionAuthority`, that a sponsor is only the recorded rent recipient and that sponsoring this authority is a non-recoverable subsidy unless the user cooperates. The program behavior matches that disclosure. `initialize_subscription_authority` records the sponsor as `SubscriptionAuthority.payer` only when the sponsor funds first creation, while `close_subscription_authority` and `revoke_subscription_authority` both route through `close_authority`, which requires the stored authority owner to match the signing user before closing and only uses the sponsor as the lamport receiver when that user supplies the recorded payer as `receiver`.

3.3.5 Transfer events report gross amount as received amount

Severity: Informational

Context: fixed_transfer.rs#L19-L24, recurring_transfer.rs#L19-L28, subscription_transfer.rs#L19-L28

Description: The transfer event schemas expose a single `amount` field and a `receiver` field. For ordinary tokens this can look like the amount received by the receiver, but that interpretation breaks for Token-2022 mints with transfer fees.

For Token-2022 mints with `TransferFeeConfig`, the value emitted in `amount` is the gross transfer amount requested from the delegator. Token-2022 debits that gross amount from the source account but credits the receiver only with the net amount after the token program withholds its fee. Existing transfer-fee tests demonstrate this behavior: a `10_000_000` transfer debits the delegator by `10_000_000` and credits the receiver with `9_900_000`.

Consequently, off-chain consumers can overstate received payments for fee-charging mints. Indexers, billing systems, merchant dashboards, settlement jobs or support tooling may treat the event as proof that the receiver obtained the full gross amount even though the receiver balance increased by less. This can produce incorrect revenue records, invoice-crediting errors or payment disputes.

The program currently also tracks allowances and billing-period pulls by the gross transfer amount. This finding does not decide whether that accounting behavior is correct. If plan or delegation amounts are intended to represent the net amount received by the merchant or delegatee, that should be handled as a separate payment-accounting issue.

Recommendation: Make transfer events explicit about gross and net amounts. Either rename and document `amount` as the debited amount and add a `net_amount_received` field or emit both the source debit and destination credit by comparing balances around the token transfer CPI. Downstream systems should use the net received amount when crediting invoices, service access or merchant balances for transfer-fee mints.

Separately decide whether configured subscription and delegation amounts are intended to mean gross amount debited or net amount received. If the intended value is net received, reject `TransferFeeConfig` mints or implement gross-up logic with a caller- or user-supplied maximum fee.

Solana Foundation: Fixed by adding some doc comments in 2ab16b2. Gross amount is what the event includes; indexers should derive net received off-chain from balances.

Cantina Managed: Fix verified. Commit 2ab16b2 changes the Rust comments on `FixedTransferEvent`, `RecurringTransferEvent` and `SubscriptionTransferEvent` to say `amount` is the gross amount debited and that net received should be derived from balances off-chain.

3.3.6 Mint validators accept accounts that are not initialized mints

Severity: Informational

Context: token.rs#L67-L79, token.rs#L151-L171

Description: `MintAccount::check` and `Mint2022Account::check` validate only the account owner, length and Token-2022 marker byte. They do not parse the account as an initialized mint. For SPL Token, an 82-byte token-program-owned account passes even if the mint state is still uninitialized. For Token-2022, an account can pass when it has the mint discriminator at byte 165, even if the underlying account is not a usable initialized mint.

`create_plan` relies on `MintInterface::check_with_program`, so a merchant can publish a plan whose `mint` field points to an account that only looks mint-shaped. Later token approval and transfer CPIs should still reject unusable mints, so this does not create direct token movement by itself.

Consequently, the plan registry can contain plans that appear to reference real token mints but cannot actually be used for token operations. Wallets, marketplaces and indexers that trust the registry may display misleading payment information or route users toward plans that fail once token-program validation is enforced.

Recommendation: Validate mint accounts by unpacking initialized mint state instead of checking only owner, length and marker bytes. Use `Pack::unpack` for SPL Token mints. For Token-2022, use `StateWithExtensions::<Mint>::unpack` or an equivalent parser that confirms the account is an initialized mint and rejects malformed extension layouts, multisigs and marker-byte false positives.

Solana Foundation: Fixed in commit 8ef1503.

Cantina Managed: Fix verified. `MintAccount::check` now rejects SPL Token mint-sized accounts unless byte 45, the shared mint `is_initialized` flag, is set to `1`; `Mint2022Account::check` now applies the same initialized check before accepting either base-length Token-2022 mints or extended mints with the mint discriminator at byte 165. `CreatePlanAccounts::try_from` still routes `create_plan` through `MintInterface::check_with_program`, so a zeroed token-program-owned account can no longer be registered as a plan mint.

3.3.7 Direct delegation expiry has 120-second spend grace

Severity: Informational

Context: transfer_validation.rs#L6-L11

Description: `is_effectively_expired` does not treat a finite delegation as expired at `expiry_ts`. Instead, it only returns expired when the current timestamp is greater than `expiry_ts + TIME_DRIFT_ALLOWED_SECS` and `TIME_DRIFT_ALLOWED_SECS` is 120 seconds.

Fixed transfers, recurring transfers and sponsor revocation all rely on this helper. As a result, a direct delegation can remain spendable for roughly two minutes after the expiry timestamp selected by the delegator. During that same window, a sponsor that funded the delegation also cannot reclaim rent based on expiry.

Consequently, `expiry_ts` is not a hard stop even though the comments and architecture documentation describe direct delegations as usable until expiry and sponsor revocation as available after expiry. A delegator may believe a delegatee can no longer spend after the displayed expiry time, while the delegatee can still submit a transfer during the drift window if allowance remains. Sponsors can also see rent recovery delayed by the same grace period.

For example, if a fixed delegation expires at 12:00:00, a transfer submitted at 12:01:30 can still pass the expiry check because the helper only treats the delegation as expired after the extra 120 seconds have elapsed.

Recommendation: Apply the clock-drift tolerance only where it is needed for timestamp validation at creation time. For transfer execution and sponsor revocation, treat direct delegations as expired once the current timestamp is greater than `expiry_ts`.

If the post-expiry spend window is intentional, document it clearly in the public docs, SDK and UI wherever users set or review `expiry_ts`.

Solana Foundation: Fixed in commit 2fd1812.

Cantina Managed: Fix verified. Commit `2fd18127f6022d07a4479a968eb1dc28fe1e8d7c` removes `TIME_DRIFT_ALLOWED_SECS` from the shared direct-delegation expiry gate. `is_expired` now returns true for finite expiries when `current_ts > expiry_ts` and fixed transfers, recurring transfers and sponsor revocation all use that hard-stop check before spending or sponsor recovery. The remaining on-chain drift tolerance is limited to creation-time timestamp validation.

3.3.8 Pausable mints let the issuer freeze all in-flight subscriptions/delegations mid-term

Severity: Informational

Context: token.rs#L151-L171

Description: Mint validation checks only program-ownership and the type-discriminator byte and inspects no extensions, so mints carrying the `Pausable` extension are accepted.

A `Pausable` mint's pause authority — the token issuer, which is a third party for any token the merchant/subscriber did not issue — can pause the mint, after which every transfer, including delegated pulls, reverts until unpaused. A third party can therefore freeze every active subscription and delegation denominated in that mint at will, with no on-chain mitigation.

Recommendation: Maintain an explicit allow-list of supported Token-2022 extensions and reject Pausable (and other issuer-controllable freeze extensions) during mint validation, or explicitly document the trust assumption placed on the mint issuer for plans/delegations denominated in such mints.

Solana Foundation: Acknowledged. T22 extension, which is by design accepted.

Cantina Managed: Acknowledged.

3.3.9 `createPlan` IDL hard-codes the SPL Token program as the `token_program` default

Severity: Informational

Context: mod.rs#L156-L160

Description: The codama attribute on `CreatePlan` makes the generated IDL default `token_program` to SPL Token. Clients relying on the default cannot create plans for Token-2022 mints.

Recommendation: Remove the hard-coded default (require the caller to supply it) or resolve it from the mint's owning program.

Solana Foundation: Acknowledged.

Cantina Managed: Acknowledged.

3.3.10 `check_and_update_version` may write the version byte before the account kind is validated

Severity: Informational

Context: core.rs#L18-L58

Description: `check_and_update_version` dispatches on `data[VERSION_OFFSET]` and (on the migration branch) writes it before the typed loader checks the discriminator.

Inert today (CURRENT_VERSION=1: fast path writes nothing; callers pre-check ownership+writability; `revoke_delegation` checks the discriminator first), but an ordering/trust-boundary hazard once a real in-place migration is added.

Recommendation: Pass the expected `AccountDiscriminator` into `check_and_update_version` and validate kind/min-length before any mutating migration step.

Solana Foundation: Fixed in commit 40d9e00.

Cantina Managed: Fix verified. `check_and_update_version` now takes an `expected: AccountDiscriminator`, rejects a mismatched discriminator before reading `VERSION_OFFSET` or entering `try_lazy_update` and all five production callers pass the concrete account kind (`FixedDelegation`, `RecurringDelegation` or `SubscriptionDelegation`).

3.3.11 Refactor replaced a self-contained `checked_mul` with an unchecked `period_length_secs()`

Severity: Informational

Context: create_plan.rs#L44-L47

Description: Commit `12728d1` replaced a locally-bounded `(period_hours as i64).checked_mul(3600)` with `PlanTerms::period_length_secs()`, an unchecked multiply whose safety now depends on `period_hours <= MAX_PLAN_PERIOD_HOURS (8760)` enforced in a *different* instruction (and on no future migration writing unbounded `period_hours`). Currently unreachable as an overflow.

Recommendation: Restore `period_hours.checked_mul(SECS_PER_HOUR).ok_or(ArithmeticOverflow)`, or re-validate the bound at load.

Solana Foundation: Fixed in commit 4497133.

Cantina Managed: Fix verified. `PlanTerms::period_length_secs` now returns `Result<u64, ProgramError>` and converts hours to seconds with `checked_mul(SECS_PER_HOUR)`, returning `SubscriptionsError::ArithmeticOverflow` on overflow instead of relying on the plan-creation bound. Both production callers that use snapshotted subscription terms, `transfer_subscription` and

`cancel_subscription`, now propagate that error with `?`, so a malformed future `period_hours` cannot overflow inside this helper.

3.3.12 `get_mint_decimals` uses `unpack_from_slice`, which panics on <82-byte mint data

Severity: Informational

Context: transfer_utils.rs#L51-L53

Description: `get_mint_decimals` calls `TokenMint::unpack_from_slice`, whose `array_ref![src, 0, 82]` **panics** on inputs <82 bytes (the `.map_err` does not catch it) and reads only the first 82 bytes of an extended mint.

Currently gated (`MintInterface::check_with_program` rejects sub-82-byte mints first; `decimals` lives at offset 44, so the read is correct), so no reachable panic but a robustness regression that replaced a self-contained bounded reader with a panic-capable one relying on an upstream invariant.

Recommendation: Read `decimals` via a bounded direct offset (as before `f25ccf7`) or use the length-checked `unpack`.

Solana Foundation: Fixed in commit 7bd3788.

Cantina Managed: Fix verified. `get_mint_decimals` now reads `data.get(MINT_DECIMALS_OFFSET)` instead of calling `TokenMint::unpack_from_slice`, so short mint data returns `InvalidAccountData` rather than panicking, while base and extended SPL Token / Token-2022 mints still read the canonical decimals byte at offset 44.

3.3.13 `emit_event` does not assert the caller-supplied `self_program` equals `crate::ID`

Severity: Informational

Context: event_engine.rs#L142-L159

Description: `emit_event` passes `program_id = &crate::ID` but takes `self_program` from caller-supplied accounts without asserting equality. Safe today (`invoke_signed` requires the program account to match, so a wrong `self_program` fails the CPI), but the explicit assertion is missing.

Recommendation: Assert `self_program.address() == &crate::ID` before the CPI for a clear, program-defined error.

Solana Foundation: Fixed in commit 2fe917c.

Cantina Managed: Fix verified. `event_engine::emit_event` now rejects `self_program.address() != &crate::ID` with `InvalidSelfProgram` before constructing or invoking the self-CPI.

3.3.14 Non-transferable mints create unusable payment rights

Severity: Informational

Context: token.rs#L151-L171

Description: `Mint2022Account::check` accepts any Token-2022 mint account with the correct owner and mint account marker. It does not parse the mint extension list or reject `NonTransferable`, even though the program already defines a `MintHasNonTransferable` error.

Consequently, users and sponsors can create live-looking subscription authorities, fixed delegations, plans and subscriptions for a mint whose token program will never allow normal transfers between token accounts. The subscriptions program records the payment right as valid, but every delegated pull fails inside Token-2022. This is different from a temporary pause: the mint's non-transferable property is permanent for the mint, so the created payment right is unusable from the start.

This can strand account rent and break billing liveness. For example, a sponsor can fund a subscription or direct delegation for a non-transferable mint, the setup transactions can all succeed and the merchant or delegatee can still be unable to collect any payment. The affected accounts then need separate cancellation, expiry, plan deletion or manual cleanup even though they could never transfer.

Recommendation: Reject unsupported Token-2022 extensions during mint validation. At minimum, parse Token-2022 mint state in `Mint2022Account::check` and return `MintHasNonTransferable` when the mint includes the `NonTransferable` extension.

If the protocol wants to support only a known-safe subset of Token-2022, make that allow-list explicit and apply it consistently in authority initialization, direct delegation creation, plan creation and transfer-time validation.

Solana Foundation: Acknowledged.

Cantina Managed: Acknowledged.

3.3.15 Memo-required Token-2022 accounts cannot receive program transfers

Severity: Informational

Context: transfer_utils.rs#L186-L210

Description: `transfer_with_delegate` invokes Token-2022 directly for delegated transfers. It does not emit a Memo-program CPI immediately before calling Token-2022 and it does not reject destination accounts that require incoming transfer memos.

Token-2022's `MemoTransfer` account extension requires an immediately preceding Memo instruction at the token CPI's invocation level. A top-level transaction memo does not reliably satisfy this requirement for a nested transfer made by the subscriptions program. Consequently, fixed, recurring and subscription transfers to otherwise valid memo-required Token-2022 accounts can fail atomically inside Token-2022.

This is a compatibility issue since a receiver can use a valid Token-2022 account configuration, but that account is unusable as a payment destination through this program.

Recommendation: Detect `MemoTransfer` on the destination token account before transferring. Either invoke the Memo program immediately before the Token-2022 transfer CPI and require the Memo program account in the transfer interface or reject memo-required destinations with a clear program error before any transfer accounting is updated.

Solana Foundation: Solved through documentation in fd4dd65 to avoid a breaking behavior/interface change.

Cantina Managed: Solved through documentation. Commit fd4dd65 updates `README.md` to state that memo-required Token-2022 destination accounts are unsupported and that the transfer fails atomically.

3.3.16 CPI Guard blocks subscription-authority initialization

Severity: Informational

Context: initialize_subscription_authority.rs#L120-L128

Description: `initialize_subscription_authority` always performs a Token-2022 `Approve` CPI when the mint uses Token-2022. This happens even if the user's token account already records the derived `SubscriptionAuthority` as the delegate with enough allowance.

Token-2022 accounts can enable the `CpiGuard` extension. When that guard is active, Token-2022 rejects `Approve` instructions executed through CPI. The user's signature on the outer subscriptions instruction does not bypass this rule because Token-2022 checks that the approval itself is being processed during CPI.

Consequently, a valid single-owner Token-2022 account with CPI Guard enabled cannot initialize or refresh a `SubscriptionAuthority` through this program while keeping the guard enabled. Re-running initialization also fails for an already-correct authority because the instruction still issues the prohibited inner approval.

Recommendation: Add a state-only initialization mode for CPI Guard accounts. When CPI Guard is enabled, require the user to approve the derived `SubscriptionAuthority` with a top-level Token-2022 instruction, then have `initialize_subscription_authority` verify the existing delegate and allowance without issuing another `Approve` CPI.

For existing authorities, skip the approval CPI when the token account already has the expected delegate and sufficient delegated amount. If the required approval is missing, fail before account creation or durable state changes and return a clear error.

Solana Foundation: Acknowledged.

Cantina Managed: Acknowledged.

3.3.17 Transfer-hook payments are only tested for one of the three payment types

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: The program can pull funds in three ways including fixed delegations, recurring delegations, and subscriptions. All three share the same code for handling Token-2022 ”transfer-hook” tokens (tokens that run an extra checker program on every transfer).

However, only the fixed-delegation path has a test that actually runs a real, switched-on hook (test_-transfer_fixed_delegation.rs:180, plus a missing-account test at :227).

The recurring path is only tested with a switched-off hook, so the hook code never actually executes. And the subscription path has no hook test at all.

Recommendation: Copy the fixed-delegation hook tests to the recurring and subscription paths: for each, add one test that completes a transfer with a live hook and one that confirms the transfer is rejected when a required hook account is missing.

Solana Foundation: Fixed in commit f276855.

Cantina Managed: Fix verified. The fix adds live Token-2022 transfer-hook coverage for both previously uncovered payment paths: `test_recurring_transfer_token_2022_active_transfer_hook` and `test_subscription_transfer_token_2022_active_transfer_hook` configure an active hook program, forward the hook program/validation/counter accounts, complete the transfer and assert the hook counter increments.

3.3.18 A mint-reading helper assumes its caller already confirmed the account is a real mint

Severity: Informational

Context: transfer_hook_util.rs#L60

Description: The helper `mint_transfer_hook_program_id` inspects a token's mint account to determine whether a transfer hook is configured.

It does this by jumping to a fixed byte offset and interpreting the data there, but it never checks that the account it was handed is actually a mint and it relies on every caller to have validated that first.

As of now this is safe, because all three transfer paths call `MintInterface::check_with_program` beforehand. However if a future caller invokes the helper without that prior check, it would read meaningless bytes from a non-mint account and could return a bogus hook program, causing confusing failures.

Recommendation: Have the helper validate the account itself before reading and confirm it is a Token-2022 mint (e.g. the mint discriminator byte is set for accounts larger than the base size) so it is safe regardless of who calls it.

Solana Foundation: Acknowledged. Caller's responsibility accepted, we want to avoid duplicating validation for CU.

Cantina Managed: Acknowledged.

3.3.19 Mutable transfer hooks can freeze existing delegations

Severity: Informational

Context: token.rs#L151-L171

Description: `Mint2022Account::check` accepts Token-2022 mints with a `TransferHook` extension without checking whether the hook program is immutable. Authority initialization, direct delegation creation, plan creation and subscription creation therefore bind user consent only to the mint pubkey. They do not bind consent to the transfer hook program or the hook authority.

`transfer_with_delegate` then reads the current hook program from the live mint before each delegated transfer. Token-2022 allows the mint's transfer-hook authority to update that program id after the mint has been initialized. Consequently, a third-party hook authority can leave the hook inactive while users create fixed delegations, recurring delegations, plans or subscriptions, then later set an arbitrary hook program or account policy. Existing payment rights can stop transferring even though the subscriptions program state has not changed.

This is a durable liveness break for delegated payments. For example, a mint issuer can activate an unusable hook after a user creates a fixed delegation. The same delegation that previously transferred successfully then fails before tokens move and subscription pullers face the same live hook lookup when collecting plan payments.

Recommendation: Reject mutable transfer-hook mints anywhere durable payment consent is created, unless this trust assumption is explicitly part of the product. At minimum, parse the Token-2022 `TransferHook` extension during mint validation and reject mints whose hook authority can still update the program id.

If mutable hooks must be supported, snapshot the accepted hook program and hook authority into the delegation or subscription consent data. Delegated transfers should reject when the live mint hook differs from the approved hook state or the UI and SDK should clearly present the hook authority as a party that can freeze future delegated payments.

Solana Foundation: Acknowledged.

Cantina Managed: Acknowledged.

3.3.20 Closeable Token-2022 mints can rebind old delegations

Severity: Informational

Context: token.rs#L151-L171

Description: `Mint2022Account::check` accepts any Token-2022 mint account with the correct owner and mint marker. It does not reject `MintCloseAuthority`, even though the program already defines a `MintHasMintCloseAuthority` error.

This matters because the program stores only the mint pubkey in long-lived consent records. `initialize_subscription_authority` approves the `SubscriptionAuthority` PDA as the user's token-account delegate and later transfers check the mint pubkey, authority fields and `init_id`. They do not prove that the live mint account is the same asset instance the user approved.

A zero-supply Token-2022 mint with `MintCloseAuthority` can be closed and later initialized again at the same address. Existing token accounts still point to that mint pubkey and their delegate fields are not tied to a mint generation. If new tokens are then placed in the existing user token account, an old fixed, recurring or subscription delegation can spend that new balance without fresh user consent for the reinitialized mint.

Recommendation: Reject Token-2022 mints with `MintCloseAuthority` in every instruction that creates durable user consent, including subscription authority initialization, direct delegation creation, plan creation and subscription creation.

The minimal fix is to parse Token-2022 mint extensions in `Mint2022Account::check` and return `MintHasMintCloseAuthority` when the extension is present. If closeable mints must be supported,

store and verify a mint-generation value that changes across close and reinitialize events before any delegated transfer succeeds.

Solana Foundation: Acknowledged.

Cantina Managed: Acknowledged.

3.3.21 Document that `u64::MAX` delegate approvals are finite

Severity: Informational

Context: initialize_subscription_authority.rs#L120-L130

Description: `initialize_subscription_authority` approves the `SubscriptionAuthority` PDA for `u64::MAX` raw token units. That is the largest SPL Token approval the program can request, but it is still a finite allowance rather than a true unlimited approval.

Each successful delegated transfer decreases the token account's `delegated_amount`. The protocol's recurring delegation accounting can reset every period, but the underlying token-program approval remains one shared counter for the same user and mint.

This is unlikely to matter for ordinary subscription amounts, especially with common 6- or 9-decimal mints. For example, `u64::MAX` represents about 18.4 billion whole tokens for a 9-decimal mint. The main concern is expectation-setting: if docs or clients describe the approval as permanently unlimited, that wording is technically inaccurate for arbitrary mints, very large cumulative transfer volume or long-lived replenished accounts.

This does not allow unauthorized spending. It also does not create a realistic near-term failure for typical token subscriptions. If the allowance ever becomes too low, the user can sign another initialization or approval transaction to refresh it.

Recommendation: Document the approval as ”maximum token-program allowance” rather than ”unlimited”. Make clear that the allowance is denominated in raw token units and can be refreshed by a user-signed approval or initialization transaction.

As a client hardening improvement, consider showing a renewal prompt if the source token account still delegates to the `SubscriptionAuthority` PDA but its remaining `delegated_amount` is below the next expected charge. This keeps the edge case understandable without presenting it as a security issue.

Solana Foundation: Fixed in commit fed0e5f.

Cantina Managed: Fix verified. The fix removes the two inaccurate `unlimited` approval statements from `docs/001-program-architecture.md` and replaces them with wording that describes the approval as near-unlimited `u64::MAX`, finite and refreshable.

3.3.22 Freeze authorities can freeze existing delegated payments

Severity: Informational

Context: token.rs#L67-L79

Description: `MintAccount::check` accepts a classic SPL Token mint when the account is owned by the SPL Token program and has the standard mint length. It does not unpack the mint state. Therefore it never checks whether the mint still has a `freeze_authority`.

That omission lets users create subscription authorities, fixed delegations, recurring delegations, plans and subscriptions for a mint whose issuer can still freeze token accounts. After the payment approval exists, the issuer can freeze the delegator or subscriber token account with the normal SPL Token `FreezeAccount` instruction. Future delegated transfers then fail inside `TransferChecked` because SPL Token rejects transfers from a frozen account.

The same missing base-mint check exists in `Mint2022Account::check`, but the classic SPL Token case is enough to trigger the issue without any Token-2022 extension. This is separate from Token-2022

`Pausable`: the issuer is not pausing a mint extension. The issuer is using the standard mint freeze authority to freeze an existing source token account.

Consequently, a third-party token issuer can halt existing delegated payments after consent has already been recorded. Merchants and delegatees can be left with live-looking payment rights that cannot collect until the issuer thaws the affected token account.

Recommendation: Unpack the mint state in `MintAccount::check` and `Mint2022Account::check`. Reject mints with a configured `freeze_authority` unless the product explicitly accepts issuer-controlled account freezes.

If these mints remain supported, show this trust assumption before users create durable payment approvals. For Token-2022, include the base mint freeze authority in the same token-policy checks used for issuer-controlled transfer restrictions.

Solana Foundation: Acknowledged. Creating subscriptions or allowance with a `freeze_authority` is considered user's risk.

Cantina Managed: Acknowledged.